

1.

Word Frequency and Top Three Words

Concept: We can use a dictionary to keep track of how many times each word appears. The keys will be the words, and the values will be their counts. We then sort the dictionary based on the values in descending order to find the top three.

Python Program:

Python

```
text = "apple banana orange apple grape banana apple orange mango"
words = text.split()

# 1. Count the frequency of each word
word_count = {}
for word in words:
    if word in word_count:
        word_count[word] += 1
    else:
        word_count[word] = 1

# 2. Sort the dictionary by values in descending order
# items() returns (key, value) pairs. We sort based on the value (x[1]).
sorted_words = sorted(word_count.items(), key=lambda x: x[1], reverse=True)

# 3. Display the top three
print("Word Frequencies:", word_count)
print("Top 3 most frequent words:")
for i in range(3):
    print(f"{sorted_words[i][0]} : {sorted_words[i][1]}")
```

Output:

Plaintext

```
Word Frequencies: {'apple': 3, 'banana': 2, 'orange': 2, 'grape': 1,
'mango': 1}
Top 3 most frequent words:
apple : 3
banana : 2
orange : 2
```

2.

Dictionary Features, Operations, and vs. Lists

Key Features of Dictionaries:

- **Key-Value Pairs:** Data is stored as pairs, making it highly readable and efficient for mapping data.
- **Mutable:** You can add, remove, or change elements after creation.
- **Unique Keys:** Keys must be unique and immutable (like strings, numbers, or tuples). Values can be duplicated.

Differences: Dictionaries vs. Lists

Feature	Dictionary	List
Access	Elements are accessed via their unique keys .	Elements are accessed via numerical index .
Structure	Stores data as {key: value} pairs.	Stores data as individual elements [item1, item2].
Order	Mappings. (Ordered by insertion in Python 3.7+).	Strictly ordered collections.

Program: Insertion, Deletion, and Lookup

Python

```
student = {"name": "John", "age": 20}

# 1. Insertion (Adding a new key-value pair)
student["branch"] = "ISE"
print("After Insertion:", student)

# 2. Lookup (Accessing a value using its key)
search_key = "name"
print(f"Lookup '{search_key}':", student.get(search_key))

# 3. Deletion (Removing a key-value pair)
del student["age"]
print("After Deletion:", student)
```

3.

Dictionary Methods: keys(), values(), items()

- **keys()**: Returns a view object containing all the keys in the dictionary.
- **values()**: Returns a view object containing all the values in the dictionary.
- **items()**: Returns a view object containing tuples of all (key, value) pairs.

Program:

Python

```
car = {"brand": "Ford", "model": "Mustang", "year": 1964}

print("Keys:", list(car.keys()))
print("Values:", list(car.values()))
print("Items:", list(car.items()))
```

4.

Aliasing vs. Copying in Dictionaries

Concept:

- **Aliasing:** Giving a new name to an existing dictionary. Both variables point to the **same memory location**. Changing one affects the other.
- **Copying:** Creating a brand **new dictionary** with the same contents. Changing the copy does not affect the original.

Program:

Python

```
original_dict = {"a": 1, "b": 2}

# ALIASING
alias_dict = original_dict
alias_dict["a"] = 99 # Modifying the alias

print("Original after Aliasing:", original_dict) # Original is changed

# COPYING
original_dict = {"a": 1, "b": 2} # Resetting
copied_dict = original_dict.copy()
copied_dict["a"] = 99 # Modifying the copy

print("Original after Copying:", original_dict) # Original remains
unchanged
print("Copied Dictionary:", copied_dict)
```

5.

Broadcasting in NumPy

Concept:

Broadcasting is a powerful mechanism in NumPy that allows mathematical operations to be performed on arrays of different shapes. NumPy automatically "broadcasts" the smaller array across the larger array so they have compatible shapes for element-by-element operations.

Program:

Python

```
import numpy as np

# 2D array (3 rows, 3 columns)
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

# 1D array (3 elements)
vector = np.array([10, 20, 30])
```

```
# Broadcasting: The 1D vector is added to each row of the 2D matrix
result = matrix + vector

print("Matrix:\n", matrix)
print("\nResult after broadcasting vector:\n", result)
```

6 & 9. Masking in NumPy (Questions 6 and 9 are identical)

Concept:

Masking (or Boolean Indexing) allows you to filter elements of a NumPy array based on a condition. The condition creates a "mask" of `True` and `False` values. When this mask is applied to the array, it extracts only the elements corresponding to `True`.

Program:

Python

```
import numpy as np

arr = np.array([10, 15, 20, 25, 30, 35])

# Create a boolean mask: True if element is greater than 20
mask = arr > 20

print("Original Array:", arr)
print("Boolean Mask:", mask)

# Apply the mask to filter the array
filtered_arr = arr[mask]
print("Filtered Array (elements > 20):", filtered_arr)
```

7.

Use of the 'with' statement in File Handling

Concept:

The `with` statement simplifies file handling by automatically taking care of resource management. When you open a file using `with`, it acts as a context manager and **automatically closes the file** as soon as the block of code inside it is finished, even if an error (exception) occurs during execution.

Program:

Python

```
# Writing to a file using 'with'
with open("sample.txt", "w") as file:
    file.write("Hello, VTU Students!\n")
    file.write("Welcome to File Handling.")
# No need to write file.close() here!
```

```
# Reading to verify
with open("sample.txt", "r") as file:
    content = file.read()
    print("File Content:\n" + content)
```

8.

NumPy Program: Matrix Operations

Question: Create a 3×3 matrix of random integers, display shape, transpose, and mean.

Program:

```
Python
import numpy as np

# Create a 3x3 matrix of random integers between 1 and 50
matrix = np.random.randint(1, 50, size=(3, 3))

print("Original Matrix:\n", matrix)

print("\n1. Shape of the matrix:", matrix.shape)

print("\n2. Transpose of the matrix:\n", matrix.T)

print("\n3. Mean of all elements:", matrix.mean())
```

10.

Binary Files vs. Text Files

Feature	Text Files	Binary Files
Content	Human-readable characters (ASCII/Unicode).	Non-readable streams of bytes (0s and 1s).
Line Endings	Uses newline characters (<code>\n</code>) to separate lines.	No concept of lines or line endings.
Examples	.txt, .py, .csv, .html	.jpg, .mp3, .exe, .dat (Pickled objects).
Modes	r, w, a	rb, wb, ab

Program Segment:

Python

```
import pickle

# --- TEXT FILE OPERATION ---
with open("data.txt", "w") as txt_file:
    txt_file.write("This is a human readable text file.")

# --- BINARY FILE OPERATION ---
# Using pickle to write a Python list as binary data
my_list = [10, 20, 30]
with open("data.bin", "wb") as bin_file:
    pickle.dump(my_list, bin_file) # Serializes into binary
```

11.

Count Lines and Words in a File

Program:

Python

```
# Setup: Creating a quick dummy file to read from
with open("testfile.txt", "w") as f:
    f.write("This is the first line.\nAnd this is the second line.")

# Actual Solution
line_count = 0
word_count = 0

with open("testfile.txt", "r") as file:
    for line in file:
        line_count += 1
        words = line.split() # Splits the line into words by spaces
        word_count += len(words)

print(f"Total Lines: {line_count}")
print(f"Total Words: {word_count}")
```

12.

Sort Contents of a Text File in Reverse Order

Program:

Python

```
# Setup: Creating a dummy file with unsorted content
with open("input.txt", "w") as f:
    f.writelines(["Zebra\n", "Apple\n", "Mango\n", "Banana\n"])

# 1. Read the contents
with open("input.txt", "r") as file:
    lines = file.readlines()
```

```
# 2. Sort the contents in reverse order
lines.sort(reverse=True)

# 3. Write to a separate file
with open("output.txt", "w") as file:
    file.writelines(lines)

# Verifying the output
print("Contents written to output.txt in reverse alphabetical order:")
with open("output.txt", "r") as file:
    print(file.read())
```